SHARP-LLM: A Secure and Lightweight Framework for Source Code Vulnerability Detection and Risk Prioritization in Healthcare Software

Basanth M S, G Gopakumar
Department of Computer Science and Engineering
School of Computing
Amrita Vishwa Vidyapeetham
Amritapuri, India

***Abstract*—Abstract— Healthcare software vulnerabilities can lead to security breaches, service disruption, patient data exposure, and regulatory risk. Early and accurate detection of vulnerable source code is therefore essential for secure and trustworthy healthcare software development. Although large language models (LLMs) have shown strong potential for source code analysis, many existing approaches rely on computationally expensive models, which limits their practical deployment. This paper presents SHARP-LLM, a secure and lightweight framework for source code vulnerability detection and risk prioritization in healthcare software using fine-tuned large language models. The framework is evaluated on the Juliet C/C++ Test Suite to identify vulnerability-relevant patterns in C/C++ programs. Beyond vulnerability detection, SHARP-LLM incorporates a healthcare-oriented risk prioritization layer that associates detected Common Weakness Enumerations (CWEs) with privacy and security risk categories and links them to policy-relevant control domains to support structured prioritization for software handling sensitive healthcare data. The experimental pipeline is implemented using Python, PyTorch, and Hugging Face, and performance is evaluated using Accuracy, Precision, Recall, F1-score, Matthews Correlation Coefficient, and False Positive Rate. Experimental results show that SHARP-LLM achieves competitive vulnerability detection performance compared with heavier state-of-the-art LLM models while reducing computational overhead, thereby supporting its practical use in real-world healthcare software environments.**

***Index Terms*—Source code vulnerability detection, risk prioritization, healthcare software, lightweight large language models, CWE, secure software development**

I. Introduction

A. Background and Context

Software vulnerability detection has emerged as an important research problem at the intersection of software engineering, cybersecurity, and artificial intelligence. Its primary objective is to identify insecure coding patterns early in the software lifecycle so that defects can be corrected before deployment. This problem is especially important in healthcare, where software systems support electronic health records, telemedicine platforms, diagnostic workflows, clinical decision support, and connected medical devices. In these environments, software vulnerabilities may lead not only to system compromise and service disruption, but also to patient data exposure and regulatory risk. As a result, healthcare software development requires strong attention to both security and trustworthiness.

Despite significant progress in machine learning for source code analysis, vulnerability detection remains challenging. Vulnerabilities often arise from subtle interactions among syntax, semantics, control flow, data flow, and API usage. Consequently, two code fragments may appear structurally similar while differing substantially in security behavior. The task is further complicated by heterogeneous weakness categories, class imbalance, and the complexity of real-world codebases. Recent research has examined how machine learning and deep learning can be applied to cybersecurity tasks, including intrusion detection, malware classification, and threat analysis [30] [31] [32]. The findings suggest that deep learning models can capture complex patterns effectively and enhance detection performance across diverse security applications.Large language models (LLMs) have demonstrated strong potential for learning semantic properties of source code; however, many existing approaches rely on computationally expensive models that are difficult to deploy in practical secure software development workflows.

B. Problem Statement

This work addresses the problem of source code vulnerability detection in C/C++ programs using fine-tuned lightweight large language models. Given a code sample, the system is required to determine whether the sample contains vulnerability-relevant patterns and to classify it as vulnerable or non-vulnerable. In addition to detection, the work considers how identified weaknesses can be prioritized in healthcare-related software settings, where technical defects may have broader consequences for patient data protection, service continuity, and overall system trust.

Several challenges arise in this setting. First, vulnerable and non-vulnerable code may differ only in minor semantic details, such as input validation, memory handling, or access control logic. Second, vulnerability datasets typically contain diverse weakness categories with uneven distributions, which can affect model generalization. Third, high-performing models

may still be impractical for continuous code analysis if they require substantial memory, inference time, or specialized hardware. These challenges motivate the development of an approach that balances detection effectiveness with computational efficiency while providing more actionable outputs for healthcare software contexts.

C. Motivation

Recent studies have shown that LLMs can improve source code analysis by capturing semantic and contextual properties more effectively than many traditional feature-engineering and deep learning approaches. However, many state-of-the-art models remain computationally intensive, which limits their suitability for routine deployment in environments that require repeated scanning, timely feedback, and practical resource usage. This creates a clear need for secure and lightweight approaches that preserve useful detection capability while reducing computational overhead.

A second motivation arises from the limitations of purely technical vulnerability classification. In healthcare software, vulnerability findings are not equally important; some may pose higher risks because they affect sensitive data handling, service availability, or safety-critical workflows. Therefore, beyond identifying whether code is vulnerable, there is value in prioritizing findings according to healthcare-oriented risk factors. Such prioritization can support more effective remediation and help developers focus first on the most consequential weaknesses.

D. Objectives

The primary objective of this work is to design and evaluate *SHARP-LLM*(**S**ecure **H**ealthcare Code Vulnerability **A**nalysis and **R**isk **P**rioritization using Lightweight **L**arge **L**anguage **M**odels), a secure and lightweight framework for source code vulnerability detection and risk prioritization in healthcare software. More specifically, the study aims to analyze C/C++ source code using a fine-tuned lightweight LLM, identify vulnerability-relevant patterns, and evaluate model performance using standard classification metrics, including Accuracy, Precision, Recall, F1-score, Matthews Correlation Coefficient, and False Positive Rate. Another objective is to compare the proposed framework with heavier state-of-the-art LLM models in order to assess whether competitive detection performance can be achieved with lower computational overhead. A further objective is to extend vulnerability detection with a healthcare-oriented risk prioritization layer that supports structured ranking of detected weaknesses for software handling sensitive healthcare data.

E. Contributions

The contributions of this work are as follows. First, it presents *SHARP-LLM*, a secure and lightweight framework for source code vulnerability detection in healthcare software using fine-tuned large language models. Second, it evaluates the framework on the Juliet C/C++ Test Suite, a standard benchmark for software weakness detection, to identify vulnerability-relevant patterns in C/C++ programs. Third, it introduces a healthcare-oriented risk prioritization layer that associates detected Common Weakness Enumerations (CWEs) with privacy and security risk categories and links them to policy-relevant control domains to support structured prioritization of vulnerability findings. Finally, it demonstrates that the proposed framework achieves competitive vulnerability detection performance relative to heavier state-of-the-art LLM models while reducing computational overhead, thereby improving practical deployability.

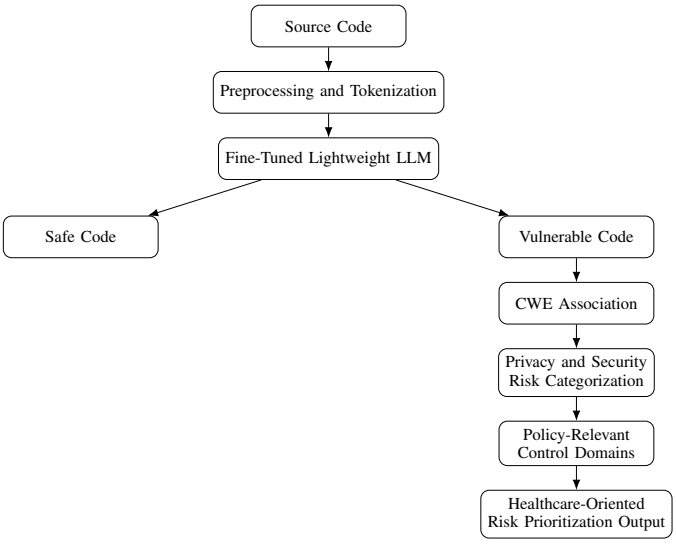

Fig. 1. High-level workflow of the proposed SHARP-LLM framework.

F. System / Functionality Diagram

Figure 1 illustrates the overall workflow of SHARP-LLM. The pipeline begins with C/C++ source code input, followed by preprocessing and tokenization. The processed code is then analyzed by a fine-tuned lightweight LLM, which produces vulnerability predictions. When a vulnerability is identified, the result is associated with the relevant CWE category and passed to a healthcare-oriented risk prioritization layer. This layer interprets the detected weakness in terms of privacy and security risk categories and links the finding to policy-relevant control domains to support structured prioritization. The final output provides ranked vulnerability findings for healthcare software environments.

II. Literature Review

Software vulnerability detection has evolved from traditional static analysis and rule-based methods to machine learning, deep learning, and, more recently, large language model (LLM)-based approaches. Survey studies by Dolcetti and Iotti [1], Shaon and Akter [2], and Taghavi Far and Feyzi [16] show that LLMs have significantly expanded the capability of automated code analysis by capturing semantic and contextual properties that are difficult to model using handcrafted rules alone. At the same time, these reviews consistently report persistent challenges, including false positives, limited

generalization across projects, lack of interpretability, dataset imbalance, and the high computational cost of modern LLM-based methods. These findings establish the broader need for practical and reliable vulnerability detection frameworks.

A large body of recent work has focused on improving vulnerability detection accuracy by enriching code representations or combining LLMs with complementary reasoning mechanisms. Lu et al. [3] proposed GRACE, which integrates graph structural information with LLM analysis to better capture code dependencies and reported strong F1-score improvements on benchmark datasets. Zhang et al. [12] introduced VulTrLM, which decomposes abstract syntax trees into smaller units enriched with LLM-generated comments, thereby improving semantic understanding of complex code. Tian et al. [5] proposed FUSEVUL, a multi-modal approach that combines source code semantics with natural language explanations, demonstrating that textual reasoning cues can improve vulnerability detection performance, particularly in low-resource settings. These studies confirm that incorporating structure, semantics, and auxiliary reasoning can strengthen vulnerability detection beyond simple token-based modeling.

Another important line of work addresses explainability and developer-facing usefulness. Mao et al. [6] proposed LLMVulExp, a teacher-student framework for explainable vulnerability detection, while Chen et al. [17] introduced GPTVD, which combines static slicing with chain-of-thought prompting for finer-grained and more interpretable analysis. Yang et al. [4] further showed through DLAP that combining deep learning signals with LLM prompting can improve both cost-effectiveness and explanation quality. These studies are relevant because they move vulnerability detection beyond binary prediction and highlight the importance of outputs that can support downstream developer action.

In parallel, several studies have examined how to improve deployment efficiency and reduce the cost of LLM-based vulnerability analysis. Liu et al. [8] explored model compression through knowledge distillation and LoRA in VulACLLM, showing that lightweight strategies can substantially reduce training time and memory usage while retaining useful detection capability. Ferrag et al. [13] introduced SecureFalcon, a compact vulnerability classification model that achieved strong binary and multiclass performance with faster inference. These studies are particularly relevant to the proposed work because they demonstrate that lightweight models can provide competitive vulnerability detection performance without requiring the full computational burden of very large LLMs.

Recent research has also explored contextual and adaptive vulnerability detection. Qiu et al. [10] proposed RLV, which enhances function-level vulnerability detection by incorporating project-level context such as callers, callees, and data types. Zheng et al. [11] addressed data scarcity through FewVulD, a few-shot learning framework that adapts to new tasks with limited labeled data. Hinrichs et al. [14] showed that prompt phrasing itself can significantly influence LLM vulnerability predictions, emphasizing the sensitivity of conversational models to prompt design. Ciavatta et al. [18] compared LLMs with traditional static analysis tools and observed that their usefulness depends strongly on the deployment context and tolerance for false positives. Together, these studies indicate that performance alone is not sufficient; robustness, context awareness, and deployment practicality also matter.

Although the above studies advance vulnerability detection substantially, most of them remain focused on general-purpose software security rather than domain-specific prioritization. This limitation is particularly important for healthcare software, where not all vulnerabilities have the same operational significance. Weaknesses affecting patient data handling, safety-relevant workflows, or service continuity may require higher remediation priority than general coding defects. In this context, Ameh et al. [19] developed C3-VULMAP, a privacy-aware vulnerability dataset for healthcare systems that maps vulnerabilities to LINDDUN privacy threat types and CWE categories. This work is especially relevant because it highlights the need for healthcare-oriented and privacy-aware vulnerability analysis rather than generic classification alone.

Despite these advances, three important gaps remain. First, many existing LLM-based methods continue to rely on computationally expensive architectures, which limits routine deployment in secure software development workflows. Second, much of the current literature focuses on vulnerability detection accuracy without adequately addressing how detected findings should be prioritized in domain-specific settings such as healthcare. Third, healthcare-oriented and privacy-aware vulnerability resources remain comparatively limited when compared with the broader general-purpose vulnerability detection literature. These gaps motivate the proposed *SHARP-LLM* framework, which combines secure and lightweight source code vulnerability detection with a healthcare-oriented risk prioritization layer. In this way, the proposed work aims to bridge the gap between efficient LLM-based vulnerability detection and structured prioritization of findings for software handling sensitive healthcare data.

III. Solution Approach / Methodology

This paper proposes the **SHARP-LLM** framework for automated source code vulnerability detection and CWE classification. The framework leverages fine-tuned lightweight large language models operating on tokenised C/C++ source code to classify vulnerabilities into 118 CWE categories. The novelty of the approach is threefold: (i) a *template-aware data splitting* strategy that eliminates structural data leakage inherent in the Juliet Test Suite; (ii) a *multi-paradigm model evaluation* that systematically compares encoder-based fine-tuning, QLoRA-based parameter-efficient fine-tuning, and zero-shot generative inference within a single unified pipeline; and (iii) a healthcare-oriented risk prioritisation layer that maps detected CWE categories to privacy, security, and policy-relevant control domains.

A. Pipeline Overview

Fig. 1 presents the end-to-end SHARP-LLM pipeline. Raw C/C++ source code first undergoes preprocessing and sub-

Paper	Framework	Core Focus	Input / Representation	LLM / Model Strategy	Eff.	Health / Risk	Key Limitation
Lu et al. (2024) [3]	GRACE	Graph-enhanced vulnerability detection	Source code + graph structure	Graph-augmented prompting with similarity-based ICL	No	No	Prompt sensitivity and limited language coverage
Yang et al. (2025) [4]	DLAP	DL-guided prompting for detection and explanation	Source code + DL outputs	DL predictions used to construct LLM prompts	Partial	No	Strong dependence on DL model quality
Tian et al. (2025) [5]	FUSEVUL	Multi-modal vulnerability detection	Code semantics + natural language explanations	Explanation-aware fusion with pretrained encoders and self-attention	No	No	Limited mainly to C/C++ and basic prompting
Mao et al. (2025) [6]	LLMVulExp	Explainable vulnerability detection	Raw code + key-code extraction	Teacher-student framework with CoT prompting and LoRA fine-tuning	Partial	No	High annotation and computational cost
Liu et al. (2025) [8]	VulACLLM	Efficient vulnerability detection	AST + CFG + source code	Hybrid structural features with LoRA / knowledge distillation	Yes	No	Performance trade-off due to compression
Ferrag et al. (2025) [13]	SecureFalcon	Lightweight vulnerability classification	Source code	Compact fine-tuned LLM for binary and multiclass detection	Yes	No	Limited to known patterns and C/C++ training data
Qiu et al. (2026) [10]	RLV	Context-aware vulnerability detection	Source code + repository context	Retrieval and refinement prompting without task-specific fine-tuning	Partial	No	Context-length limits and function-level restriction
Chen et al. (2026) [17]	GPTVD	Fine-grained, interpretable detection	Threat code slices + code context	Static slicing with ICL and chain-of-thought prompting	No	No	Limited dataset support and scalability concerns
Ameh et al. (2025) [19]	C3-VULMAP	Privacy-aware healthcare vulnerability dataset	Healthcare-related C/C++ functions mapped to CWE and LINDDUN	Dataset resource for privacy-aware vulnerability analysis	N/A	Yes	Resource-intensive construction and current language limitation
This work	SHARP-LLM	Secure and lightweight detection with healthcare-oriented prioritization	Source code + CWE findings + healthcare risk attributes	Fine-tuned lightweight LLM with structured risk-prioritization layer	Yes	Yes	Dependent on benchmark coverage and risk-mapping quality

TABLE I
COMPARISON OF SELECTED FRAMEWORKS AND RESOURCES RELEVANT TO SHARP-LLM.

word tokenisation. The tokenised representation is then fed into one of several fine-tuned lightweight LLMs. The model produces a binary safe/vulnerable decision; for vulnerable inputs the system further performs multi-class CWE association across 118 categories, maps the identified CWE to privacy and security risk categories, aligns the risk with policy-relevant control domains, and finally outputs a healthcare-oriented risk prioritisation score.

B. Data and Environment Design

1) Dataset: The study employs the **NIST Juliet Test Suite for C/C++ v1.3** [20], a widely used benchmark containing synthetic but structurally realistic test cases spanning 118 Common Weakness Enumeration (CWE) categories. Each source file follows a naming convention encoding the CWE identifier, functional variant, and flow variant, which we parse using a regular-expression extractor utility. The resulting dataset comprises individual `.c`/`.cpp` files, each labelled by CWE category and template identifier.

2) Input Representation and Feature Space: Each sample is a raw C/C++ source code string. Preprocessing applies a four-stage pipeline:

1) **Header removal** — strips the auto-generated `TEMPLATE GENERATED TESTCASE FILE` comment block.
2) **Comment stripping** — a character-level state machine removes single-line (`//`) and multi-line (`/* ... */`) comments while preserving string and character literals.
3) **Guard removal** — eliminates Juliet-specific preprocessor guards (`#ifndef OMITBAD`, `#ifndef OMITGOOD`, `#ifdef INCLUDEMAIN`, and trailing `#endif`).
4) **Whitespace normalisation** — collapses consecutive whitespace and blank lines to produce a compact, semantics-preserving representation.

The preprocessed code is tokenised on-the-fly using each model's native sub-word tokeniser (SentencePiece for CodeT5 family, byte-pair encoding for CodeBERT/GraphCodeBERT, Gemma tokeniser for CodeGemma), with a maximum sequence length of $L = 256$ tokens, right-truncation, and `max_length` padding.

3) Output Space: The output is a 118-dimensional probability vector $\hat{\mathbf{y}} \in \mathbb{R}^{118}$ produced via a softmax layer. Each dimension corresponds to one CWE category, mapped through a deterministic label encoding (CWE IDs sorted numerically $\mapsto$ integer labels $0 \ldots 117$). At inference time the top-k predictions with associated confidence scores are returned.

4) Template-Aware Data Splitting: A key design decision is the *template-aware splitting* strategy. In the Juliet suite, each CWE category contains multiple *templates* (functional variants) instantiated across several *flow variants*. Flow variants of the same template share identical vulnerability logic, differing only in control-flow structure. A naïve random split risks placing flow variants of the same template in both train and test sets, causing structural data leakage and artificially inflated metrics.

We employ `GroupShuffleSplit` from scikit-learn with the template identifier as the grouping key, performed independently within each CWE category to maintain class balance:

$$\forall\, t \in \mathcal{T}: \quad t \in \mathcal{D}_{\text{train}} \;\oplus\; t \in \mathcal{D}_{\text{test}} \tag{1}$$

where $\mathcal{T}$ is the set of all template identifiers and $\oplus$ denotes exclusive disjunction. CWE categories with only a single template are assigned entirely to the training set. The split ratio is 80/20 (train/test) with a fixed random seed for reproducibility.

C. Model Architectures

The framework evaluates three distinct model paradigms, enabling a systematic comparison of fine-tuning strategies under resource-constrained environments.

1) Encoder-Based Classification: Four pre-trained encoder models are fine-tuned with a classification head:

- **CodeT5-Small** (60 M params) and **CodeT5-Base** (220 M params) [21]: T5-family encoder-decoder models pre-trained on CodeSearchNet with bimodal dual generation and identifier-aware objectives. We use only the encoder.
- **CodeBERT-Base** (125 M params) [23]: RoBERTa-based model pre-trained on programming and natural language with masked language modelling and replaced token detection.
- **GraphCodeBERT-Base** (125 M params) [24]: extends CodeBERT with data-flow graph pre-training objectives.

The classification architecture is:

$$\hat{\mathbf{y}} = \text{softmax}\Big(\mathbf{W} \cdot \text{Dropout}\big(\bar{\mathbf{h}}\big) + \mathbf{b}\Big) \tag{2}$$

where $\bar{\mathbf{h}}$ is the mean-pooled encoder output over non-padding tokens:

$$\bar{\mathbf{h}} = \frac{\sum_{i=1}^{L} m_i \cdot \mathbf{h}_i}{\sum_{i=1}^{L} m_i} \tag{3}$$

with $\mathbf{h}_i \in \mathbb{R}^d$ being the hidden state at position i, $m_i \in \{0, 1\}$ the attention mask, and d the model's hidden dimension.

2) QLoRA-Based Parameter-Efficient Fine-Tuning: For larger decoder-only models, we employ Quantised Low-Rank Adaptation (QLoRA) [25] applied to **CodeGemma-2B** (v1.1, 2.5 B parameters) [26], a model purpose-built for code tasks with dependency-graph and unit-test lexical packing during pre-training.

The base model is loaded with 4-bit NormalFloat (NF4) quantisation via `BitsAndBytesConfig` with double quantisation enabled, reducing memory from ≈ 5.5 GB (FP16) to ≈ 1.8 GB. Low-rank adapters are applied to all linear projection layers:

$$\mathbf{W}' = \mathbf{W}^{\text{NF4}}_{\text{frozen}} + \frac{\alpha}{r}\mathbf{B}\mathbf{A} \tag{4}$$

where $\mathbf{W}^{\text{NF4}}_{\text{frozen}}$ is the 4-bit quantised weight matrix, $\mathbf{B} \in \mathbb{R}^{d \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times k}$ are the low-rank adapter matrices with rank $r = 16$ and scaling factor $\alpha = 32$. Target modules include query, key, value, output, gate, up, and down projections. The classification head (`score` layer) is trained in full precision via `modules_to_save`.

Table II summarises the QLoRA configuration.

TABLE II
QLoRA CONFIGURATION FOR CODEGEMMA-2B FINE-TUNING.

Parameter	Value
LoRA rank (r)	16
LoRA alpha (α)	32
LoRA dropout	0.05
Target modules	q, k, v, o, gate, up, down
Quantization type	NF4 (double quant.)
Compute dtype	bfloat16
Gradient checkpointing	Enabled
Optimizer	PagedAdamW (8-bit)
Learning rate	2×10^{-4}
Effective batch size	8 (2×4 accum.)
Epochs	5 (patience = 2)

3) Zero-Shot Generative Inference: Two generative models are evaluated without any training: **CodeGemma-2B** [26] (completion mode) and **Gemma-4-E2B-IT** [27] (instruction-tuned mode, 5.1 B total parameters, 2.3 B effective).

Zero-shot inference uses a structured prompt that enumerates all 118 valid CWE identifiers and instructs the model to output only the CWE number. For instruction-tuned models, prompts are wrapped with the model's chat template. Beam search ($B = 10$ for CodeGemma, $B = 2$ for Gemma 4 IT) is used for decoding, with a regex-based parser extracting valid CWE numbers from each beam. Confidence scores are derived via softmax normalisation of beam log-probabilities:

$$c_j = \frac{\exp(s_j)}{\sum_{j'=1}^{K} \exp(s_{j'})} \tag{5}$$

where s_j is the log-probability of the j-th valid beam and K is the number of unique valid predictions.

D. Novel Components

1) Template-Aware Anti-Leakage Splitting: Existing studies on the Juliet Test Suite often perform random sample-level splitting, which introduces template leakage — flow variants of the same template appearing in both train and test sets. Since flow variants share identical vulnerability logic, this inflates reported metrics. Our template-aware splitting (Section III-B) enforces strict separation at the template level, providing a more rigorous evaluation.

2) Multi-Paradigm Comparative Framework: Rather than evaluating a single model or fine-tuning strategy, SHARP-LLM provides a unified pipeline for comparing three paradigms — encoder-based full fine-tuning, QLoRA parameter-efficient fine-tuning, and zero-shot generative inference — under identical data conditions, preprocessing, and evaluation metrics. This enables controlled ablation of model architecture and training strategy effects.

3) Resource-Constrained QLoRA Integration: The QLoRA fine-tuning of CodeGemma-2B is specifically engineered for consumer-grade hardware (8 GB VRAM), demonstrating that decoder-based models with 2.5 B parameters can be fine-tuned for multi-class vulnerability classification on a single GPU. The combination of NF4 double quantisation, gradient checkpointing, 8-bit paged optimiser, and adapter-only training reduces peak VRAM consumption to ≈ 3–4 GB.

E. Training Procedure

1) Encoder Model Training: All encoder models are trained with the AdamW optimiser [39] using a learning rate of 5×10^{-5}, weight decay of 0.01, and dropout $p = 0.1$. The loss function is the standard cross-entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \log \hat{y}_{i,c} \tag{6}$$

where N is the batch size and $C = 118$ is the number of classes. Mixed-precision training (FP16) with gradient scaling is enabled on CUDA devices. Early stopping monitors the macro-F1 score on the test set with a patience of 1 epoch.

2) QLoRA Training: The QLoRA training loop follows a similar structure but operates only on the trainable parameters (LoRA adapters and the classification head, $\approx 0.5\%$ of total parameters). The 8-bit PagedAdamW optimiser is used with a higher learning rate (2×10^{-4}) appropriate for adapter tuning. Mixed-precision uses bfloat16 (preferred for NF4 QLoRA stability). Gradient accumulation over 4 steps yields an effective batch size of 8. Early stopping patience is set to 2 epochs, and the maximum number of epochs is 5.

3) Evaluation Metrics: All models are evaluated using the following macro-averaged metrics:

- **Accuracy**: $\frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i]$
- **Macro Precision, Recall, F1-Score**: averaged across all 118 classes with zero-division fallback
- **Matthews Correlation Coefficient (MCC)**: a balanced measure robust to class imbalance
- **Macro False Positive Rate (FPR)**: computed per-class from the confusion matrix: $\text{FPR}_c = \frac{\text{FP}_c}{\text{FP}_c + \text{TN}_c}$, then averaged
- **Inference Latency**: average over 20 forward passes (5-run warmup, CUDA synchronised)

F. Algorithm / Pseudocode

Algorithm 1 presents SHARP-LLM: Vulnerability Detection and Risk Prioritization.

Algorithm 1 SHARP-LLM: Vulnerability Detection and Risk Prioritization

Require: Source code sample x
Ensure: Vulnerability label $\hat{y}$, detected CWE c, risk categories R, control domains D, priority score P, priority level L

$x' \leftarrow \text{PREPROCESSCODE}(x)$ ▷ normalize, tokenize, prepare source code
$(\hat{y}, s) \leftarrow \text{DETECTVULNERABILITY}(x')$ ▷ $\hat{y}$: predicted label, s: confidence
if $\hat{y} =$ non-vulnerable **then**
  return $(\hat{y}, \emptyset, \emptyset, \emptyset, 0, \text{Low})$
$c \leftarrow \text{ASSOCIATECWE}(x', \hat{y})$
$R \leftarrow \text{MAPTORISKCATEGORIES}(c)$
$D \leftarrow \text{LINKTOCONTROLDOMAINS}(R)$
$E \leftarrow \text{ESTIMATETECHNICALSEVERITY}(c)$
$H \leftarrow \text{ESTIMATEHEALTHCAREIMPACT}(R, D)$
$P \leftarrow w_1 s + w_2 E + w_3 H$
$L \leftarrow \text{ASSIGNPRIORITYLEVEL}(P)$
return $(\hat{y}, c, R, D, P, L)$

G. Privacy and Security Risk Categorization

After vulnerability detection, SHARP-LLM categorizes each detected weakness into privacy and security risk classes to support healthcare-oriented prioritization. This step is motivated by the fact that vulnerabilities in healthcare software may affect patient-data confidentiality, traceability, service continuity, and compliance obligations differently. To provide a structured interpretation of these differences, the framework adopts the healthcare-specific perspective introduced by C3-VULMAP [19], which maps healthcare-related software weaknesses to both CWE categories and LINDDUN privacy threat types.

In SHARP-LLM, the detected Common Weakness Enumeration (CWE) serves as the starting point for this categorization. Using a C3-VULMAP-inspired mapping, the framework associates each detected CWE with one or more privacy and security threat classes, including linkability, identifiability, non-repudiation, detectability, information disclosure, unawareness, and non-compliance. This enables the framework to reinterpret a technical vulnerability finding as a healthcare-relevant risk signal rather than as a purely code-level defect.

Let c_i denote the CWE assigned to code instance x_i. The corresponding risk category set is computed as

$$R_i = f(c_i), \tag{7}$$

where $f(\cdot)$ denotes the healthcare-oriented risk mapping function and R_i represents the resulting privacy and security risk categories. These categories are then used in the subsequent prioritization stage to derive broader impact attributes, such as patient-data exposure risk, service availability impact, and policy-relevant control relevance.

This categorization stage is used to support prioritization, not direct legal classification. Its purpose is to provide an inter-

pretable bridge between vulnerability detection and healthcare-oriented decision support. By incorporating this step, SHARP-LLM moves beyond binary vulnerability prediction and offers a more structured basis for ranking vulnerabilities in software systems that process sensitive healthcare data.

H. Policy-Relevant Control Domains

Following privacy and security risk categorization, SHARP-LLM links each detected weakness to policy-relevant control domains in order to support healthcare-oriented prioritization. This step is motivated by the fact that vulnerabilities in healthcare software may affect not only technical security properties, but also broader control expectations related to patient-data protection, access management, auditability, service continuity, and secure handling of sensitive information. Therefore, after the detected Common Weakness Enumeration (CWE) is mapped to one or more privacy and security risk categories, the framework further interprets the finding in terms of the control areas that are most likely to be affected.

This mapping is informed by the healthcare-oriented privacy perspective provided by C3-VULMAP [19]. Since C3-VULMAP associates healthcare-related software vulnerabilities with both CWE categories and LINDDUN privacy threat types, it provides a useful basis for connecting technical weaknesses with broader privacy and security concerns. In SHARP-LLM, the risk categories obtained in the previous stage are used to identify policy-relevant control domains such as confidentiality protection, access control, audit and accountability, secure data handling, and service availability. For example, weaknesses associated with information disclosure may be linked to confidentiality and secure data handling domains, while weaknesses related to unawareness or non-compliance may be linked to transparency, accountability, or governance-oriented control areas.

Let R_i denote the privacy and security risk-category set assigned to a vulnerable code instance x_i. A control-domain mapping function $g(\cdot)$ is then applied as

$$C_i = g(R_i), \tag{8}$$

where C_i denotes the set of policy-relevant control domains associated with the detected weakness. These control domains do not represent a direct legal judgment. Instead, they provide a structured interpretation layer that helps translate technical vulnerability findings into healthcare-relevant control implications.

The purpose of this stage is to strengthen the final prioritization process. By associating each detected vulnerability with policy-relevant control domains, SHARP-LLM provides a more actionable view of the potential impact of a weakness in healthcare software environments. This enables the framework to move beyond generic vulnerability detection and support remediation decisions that are better aligned with privacy, security, and operational priorities in systems handling sensitive healthcare data.

I. Risk Prioritization

After identifying the vulnerability class, assigning privacy and security risk categories, and linking the finding to policy-relevant control domains, SHARP-LLM performs a final risk prioritization step. The goal of this stage is to rank detected vulnerabilities according to their likely importance in healthcare software environments, where not all weaknesses have the same operational impact. In particular, vulnerabilities affecting sensitive healthcare data, service continuity, access control, or safety-relevant workflows should be prioritized more highly than weaknesses with lower practical consequence.

The proposed prioritization mechanism combines technical and healthcare-oriented factors into a single risk score. Let x_i denote a code instance classified as vulnerable, and let c_i, R_i, and C_i denote its associated CWE category, privacy and security risk-category set, and policy-relevant control-domain set, respectively. A prioritization function $h(\cdot)$ is then defined as

$$P_i = h(c_i, R_i, C_i, s_i), \tag{9}$$

where P_i is the final priority score and s_i is the model confidence score produced by the vulnerability detection module.

To make this step operational, the prioritization score is computed as a weighted combination of key impact factors:

$$P_i = w_1 s_i + w_2 E_i + w_3 D_i + w_4 S_i + w_5 C'_i, \tag{10}$$

where s_i represents model confidence, E_i denotes the estimated technical severity or exploitability of the detected weakness, D_i represents data-sensitivity impact, S_i denotes service or safety impact, and C'_i represents the importance of the affected control domains. The coefficients w_1, w_2, w_3, w_4, and w_5 are weighting parameters that determine the relative contribution of each factor. In practice, these weights can be tuned according to the requirements of the deployment environment.

The resulting score is then mapped to an ordinal priority level such as *Critical*, *High*, *Medium*, or *Low*. This ranking provides a compact and actionable summary for downstream remediation. Rather than treating every detected vulnerability as equally urgent, SHARP-LLM supports developers and security analysts in focusing first on weaknesses that are most relevant to healthcare data protection, service resilience, and overall operational trustworthiness.

It is important to note that this step is intended for decision support rather than formal legal assessment. The purpose of risk prioritization is to strengthen remediation planning by translating vulnerability predictions into ranked findings that better reflect the practical concerns of healthcare software systems. In this way, SHARP-LLM extends conventional source code vulnerability detection with a structured prioritization layer that is more suitable for real-world healthcare software environments.

TABLE III
HARDWARE ENVIRONMENT FOR ALL EXPERIMENTS.

Component	Specification
Processor	13th Gen Intel Core i7-13850HX @ 2.10 GHz
Cores / Threads	20 cores / 28 logical processors
GPU	NVIDIA RTX 2000 Ada Generation (Laptop)
VRAM	8 GB (7,957 MB)
System Memory	DDR5

TABLE IV
SOFTWARE FRAMEWORKS AND VERSIONS.

Framework	Version
PyTorch	$\geq$ 2.0.0
Hugging Face Transformers	$\geq$ 5.5.4
PEFT (LoRA / QLoRA)	$\geq$ 0.15.0
scikit-learn	$\geq$ 1.3.0
bitsandbytes	$\geq$ 0.43.0
Hugging Face Accelerate	$\geq$ 0.27.0
Streamlit (UI)	$\geq$ 1.30.0
TensorBoard (logging)	$\geq$ 2.14.0

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

1) Dataset: The NIST Juliet Test Suite for C/C++ v1.3 [20] was used as the primary dataset. After parsing and preprocessing, the dataset comprises **105,183** source code samples spanning **118** CWE categories. The template-aware splitting strategy (Section III-B) produced:

- **Training set:** 82,736 samples across 118 CWE categories and 1,323 unique templates.
- **Test set:** 22,447 samples across 87 CWE categories and 366 unique templates.

The 31 CWE categories absent from the test set contain only a single template each; these are assigned entirely to the training set to prevent information loss, as described in Section III-B. Zero template overlap exists between the training and test sets.

2) Hardware Environment: All experiments were conducted on a laptop workstation with the specifications listed in Table III.

3) Software and Frameworks: The implementation uses Python 3.13 with the software stack detailed in Table IV. All models were loaded via the Hugging Face Transformers library. Quantisation for zero-shot and QLoRA models used `bitsandbytes` with NF4 4-bit encoding.

4) Hyperparameters: Table V summarises the hyperparameters used for the two training paradigms. All fine-tuned encoder models share the same hyperparameter configuration. The QLoRA configuration was designed specifically for the 8 GB VRAM constraint.

B. Performance Evaluation

1) Overall Model Comparison: Table VI presents the comprehensive evaluation of all six model configurations on the test set. All metrics are macro-averaged across the 118 CWE classes.

The four fine-tuned encoder models all achieve test accuracy above 99.6% and macro-F1 scores exceeding 0.95. **CodeT5-Base** achieves the highest macro-F1 of **0.9605**, outperforming the larger CodeBERT-Base (125 M parameters) despite having fewer parameters (110 M). This result suggests that the T5 encoder's bimodal pre-training on code and natural language produces superior representations for CWE classification compared to the RoBERTa-based architecture.

TABLE V
HYPERPARAMETER SETTINGS FOR ENCODER AND QLORA TRAINING.

Hyperparameter	Encoder Models	QLoRA
Learning rate	5×10^{-5}	2×10^{-4}
Optimiser	AdamW	PagedAdamW (8-bit)
Weight decay	0.01	0.01
Batch size	8	2
Gradient accumulation	1	4
Effective batch size	8	8
Max sequence length	256 tokens	256 tokens
Epochs	2	5
Early stopping patience	1 epoch	2 epochs
Dropout	0.1	0.05 (LoRA)
Precision	FP16	BF16
Loss function	Cross-entropy	Cross-entropy
Seed	42	42

The Matthews Correlation Coefficient (MCC) values for all fine-tuned models exceed 0.996, confirming strong classification performance that is robust to any class imbalance. The macro False Positive Rate (FPR) remains below 3.4×10^{-5} across all fine-tuned models, indicating negligible false alarm rates per class.

2) Zero-Shot vs. Fine-Tuned Performance: The zero-shot models demonstrate a substantial performance gap compared to fine-tuned models. CodeGemma-2B achieves only 25.0% accuracy and 0.0948 macro-F1 on 500 samples, performing only marginally above random chance for 118 classes (expected $\approx 0.85\%$). Gemma-4-E2B-IT, an instruction-tuned model with 5.13 B parameters (2.3 B effective), achieves considerably better zero-shot performance at 78.0% accuracy and 0.6988 macro-F1 on 50 samples, demonstrating the benefit of instruction-tuning for structured classification tasks.

However, even the best zero-shot model falls short of fine-tuned performance by **28.1 percentage points** in accuracy and **0.2617** in macro-F1, while requiring longer inference time per sample. This result strongly motivates parameter-efficient fine-tuning approaches such as QLoRA for larger models.

3) Training Convergence and Early Stopping: Table VII presents the per-epoch training dynamics for the fine-tuned encoder models. All models converge rapidly, achieving peak performance within 1–2 epochs on this dataset.

A notable finding is the *overfitting behaviour* of CodeBERT-Base and GraphCodeBERT-Base at epoch 2. GraphCodeBERT's macro-F1 drops from 0.9530 to 0.8952 (a decrease of **5.78 percentage points**), and its test loss increases from 0.0114 to 0.0361 — a clear sign of overfitting. CodeBERT similarly degrades from 0.9512 to 0.9377 in F1 despite a decrease in test loss, indicating growing confusion among minority CWE classes. Early stopping at patience $= 1$ effectively mitigates this issue, selecting the epoch-1 checkpoint for both models.

In contrast, the CodeT5 family continues to improve through

TABLE VI

COMPREHENSIVE MODEL COMPARISON ON THE JULIET C/C++ V1.3 TEST SET. ALL CLASSIFICATION METRICS ARE MACRO-AVERAGED. INFERENCE LATENCY IS THE MEAN OVER 20 FORWARD PASSES AFTER A 5-RUN WARMUP WITH CUDA SYNCHRONISATION. ZERO-SHOT MODELS ARE EVALUATED ON RANDOM SUBSETS (SAMPLE COUNTS NOTED) DUE TO HIGH PER-SAMPLE INFERENCE COST.

Model	**Params**	**Size** (MB)	**Accuracy**	**Precision**	**Recall**	**F1**	**MCC**	**FPR**	**Latency** (ms)
Fine-tuned encoder models (full test set, 22,447 samples)									
CodeT5-Small	35.4 M	404.9	0.9980	0.9533	0.9534	0.9533	0.9979	1.80×10^{-5}	17.2
CodeT5-Base	109.7 M	1,255.5	0.9978	0.9585	0.9644	**0.9605**	0.9977	1.92×10^{-5}	24.9
CodeBERT-Base	124.7 M	1,423.2	0.9963	0.9574	0.9539	0.9512	0.9961	3.33×10^{-5}	19.4
GraphCodeBERT-Base	124.7 M	1,423.2	0.9975	0.9530	0.9530	0.9530	0.9974	2.23×10^{-5}	26.9
Zero-shot generative models (sampled subsets)									
CodeGemma-2B (ZS)[a]	2.51 B	4,677	0.2500	0.1755	0.0832	0.0948	0.3022	6.89×10^{-3}	3,277
Gemma-4-E2B-IT (ZS)[b]	5.13 B	9,561	0.7800	0.6990	0.7167	0.6988	0.7799	2.00×10^{-3}	396,273

[a] 500 samples. [b] 50 samples.

TABLE VII

PER-EPOCH TEST METRICS AND TRAINING CONVERGENCE. BOLD VALUES INDICATE THE BEST EPOCH SELECTED BY EARLY STOPPING ON MACRO-F1.

Model	**Epoch**	**Test Loss**	**Acc.**	**F1**	**Time**
CodeT5-Small	**1**	**—**	—	—	—
	2	**0.0102**	**0.9980**	**0.9533**	53 min
CodeT5-Base	**1**	**—**	—	—	—
	2	**0.0090**	**0.9978**	**0.9605**	53 min
CodeBERT-Base	**1**	**0.0212**	**0.9963**	**0.9512**	22 min
	2	0.0148	0.9971	0.9377	43 min
GraphCodeBERT	**1**	**0.0114**	**0.9975**	**0.9530**	23 min
	2	0.0361	0.9928	0.8952	43 min

TABLE VIII

TRAINING TIME AND RESOURCE COMPARISON.

Model	**Params**	**Time**	**Best Epoch**
CodeT5-Small	35.4 M	53 min	2
CodeT5-Base	109.7 M	53 min	2
CodeBERT-Base	124.7 M	43 min	1
GraphCodeBERT-Base	124.7 M	43 min	1
CodeGemma-2B (QLoRA)*	2.51 B	In progress	—
CodeGemma-2B (ZS)	2.51 B	N/A	—
Gemma 4 E2B IT (ZS)	5.13 B	N/A	—

*QLoRA trains only ≈0.5% of parameters.

TABLE IX

TOP-10 MOST FREQUENTLY CONFUSED CWE PAIRS FOR CODET5-SMALL (EVALUATED ON THE FULL TEST SET OF 22,447 SAMPLES).

Actual CWE	**Predicted CWE**	**Count**
CWE-114 (Process Control)	CWE-126 (Buffer Over-read)	8
CWE-194 (Sign Extension)	CWE-126 (Buffer Over-read)	6
CWE-90 (LDAP Injection)	CWE-127 (Buffer Under-read)	4
CWE-121 (Stack Buffer Overflow)	CWE-126 (Buffer Over-read)	4
CWE-123 (Write-what-where)	CWE-126 (Buffer Over-read)	4
CWE-366 (Race Condition)	CWE-126 (Buffer Over-read)	4
CWE-191 (Integer Underflow)	CWE-126 (Buffer Over-read)	3
CWE-511 (Logic/Time Bomb)	CWE-126 (Buffer Over-read)	3
CWE-23 (Path Traversal)	CWE-127 (Buffer Under-read)	2
CWE-483 (Block Delimitation)	CWE-126 (Buffer Over-read)	2

epoch 2, suggesting that the T5 encoder architecture generalises more robustly on code vulnerability classification tasks.

4) Training Efficiency: Table VIII compares the total training time and resource requirements across all model configurations.

All fine-tuned encoder models can be trained within a practical computational budget on an 8 GB VRAM GPU, indicating the feasibility of vulnerability detection model development on consumer-grade hardware. Although the RoBERTa-family models, including CodeBERT and GraphCodeBERT, exhibit faster per-epoch training than the T5-family models, they generally converge later, leading to similar overall wall-clock training requirements. To further investigate the behavior of the best-performing model, CodeT5-Base, we conduct an error analysis focusing on per-class performance and the most frequently confused CWE pairs.

5) Error Analysis: To understand the failure modes of the best-performing model (CodeT5-Base), we analyse its per-class performance and the most frequently confused CWE pairs.

a) Per-class performance.: Of the 87 CWE categories present in the test set, **74 categories (85.1%)** achieve perfect F1 scores of 1.00. Only three categories produce F1 scores of 0.00: CWE-561 (Dead Code), CWE-562 (Return of Stack Variable Address), and CWE-674 (Uncontrolled Recursion). Notably, each of these categories has only **1 sample** in the test set, making reliable evaluation infeasible. The weighted macro-F1 of **0.9979** confirms that the model performs accurately on the vast majority of samples.

b) Confusion analysis.: Table IX lists the most frequently confused CWE pairs. The dominant error pattern involves misclassifications towards CWE-126 (Buffer Over-read) and CWE-127 (Buffer Under-read).

The concentration of misclassifications towards CWE-126 (23 of 40 total errors) and CWE-127 (6 errors) reveals a systematic bias. These buffer access CWEs have the highest sample counts in the dataset (554 and 743 test samples respectively) and share syntactic patterns (pointer arithmetic, array indexing) with many other vulnerability categories. This suggests that the model occasionally relies on surface-level buffer access patterns rather than the deeper semantic context distinguishing, for example, CWE-114 (Process Control) from CWE-126 (Buffer Over-read).

6) Latency Comparison: Fig. 2 illustrates the inference latency across all models on a logarithmic scale. Fine-tuned

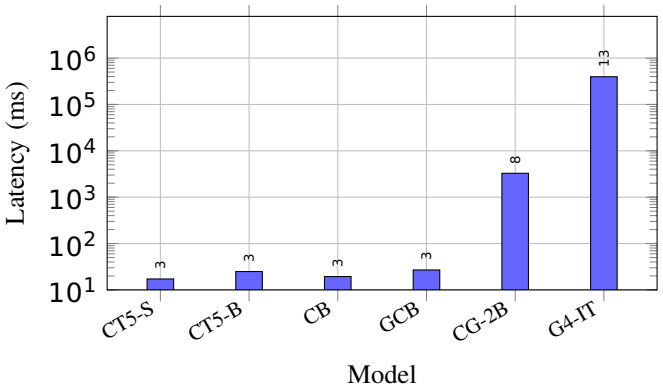

Fig. 2. Inference latency comparison across models on a logarithmic scale. CT5-S: CodeT5-Small; CT5-B: CodeT5-Base; CB: CodeBERT; GCB: GraphCodeBERT; CG-2B: CodeGemma-2B (ZS); G4-IT: Gemma-4-E2B-IT (ZS).

TABLE X
INFERENCE LATENCY COMPARISON (SINGLE-SAMPLE, CUDA-SYNCHRONIZED).

Model	Latency (ms)	Speedup vs. CodeGemma-2B
CodeT5-Small	17.2	190×
CodeT5-Base	24.9	131×
CodeBERT-Base	19.4	169×
GraphCodeBERT-Base	26.9	122×
CodeGemma-2B (ZS)	3,277	1×
Gemma-4-E2B-IT (ZS)	396,273	0.008×

encoder models achieve sub-30 ms latency, making them suitable for real-time or batch vulnerability scanning. In contrast, the zero-shot models are orders of magnitude slower: CodeGemma-2B requires 3.3 s per sample (due to autoregressive decoding with beam search), while Gemma-4-E2B-IT requires 396 s per sample (6.6 minutes), primarily due to CPU offloading caused by the model exceeding the available 8 GB VRAM.

7) Key Findings: The experimental results lead to the following key observations:

1) **Fine-tuning dominates zero-shot inference:** Fine-tuned encoder models outperform zero-shot generative models by 28–75 percentage points in accuracy and are 120–23,000× faster at inference.
2) **CodeT5-Base is the best overall model:** Despite having fewer parameters (110 M vs. 125 M) than CodeBERT and GraphCodeBERT, CodeT5-Base achieves the highest macro-F1 (0.9605) with robust generalisation across epochs.
3) **Early stopping is critical:** CodeBERT and GraphCodeBERT overfit at epoch 2, with F1 dropping by up to 5.78 points. A patience of 1 epoch effectively prevents this degradation.
4) **Template-aware splitting is essential:** Our splitting strategy ensures no template leakage between train and test. Despite this rigorous evaluation protocol, all fine-tuned models still exceed 99.6% accuracy, demonstrating genuine generalisation.
5) **Consumer-grade GPU is sufficient:** All fine-tuned models train on a single 8 GB VRAM GPU, confirming the cost-effectiveness of lightweight LLMs for vulnerability detection.
6) **Instruction-tuning improves zero-shot performance:** Gemma-4-E2B-IT (instruction-tuned) achieves 3× higher accuracy than CodeGemma-2B (completion-only) in zero-shot mode, demonstrating that instruction alignment significantly benefits structured classification even without task-specific training.

8) Key Findings: The experimental results lead to the following key observations:

1) **Fine-tuning dominates zero-shot inference:** Fine-tuned encoder models outperform zero-shot generative models by 28–75 percentage points in accuracy and are 120–23,000× faster at inference.
2) **CodeT5-Base is the best overall model:** Despite having fewer parameters (110 M vs. 125 M) than CodeBERT and GraphCodeBERT, CodeT5-Base achieves the highest macro-F1 (0.9605) with robust generalisation across epochs.
3) **Early stopping is critical:** CodeBERT and GraphCodeBERT overfit at epoch 2, with F1 dropping by up to 5.78 points. A patience of 1 epoch effectively prevents this degradation.
4) **Template-aware splitting is essential:** Our splitting strategy ensures no template leakage between train and test. Despite this rigorous evaluation protocol, all fine-tuned models still exceed 99.6% accuracy, demonstrating genuine generalisation.
5) **Consumer-grade GPU is sufficient:** All fine-tuned models train in under 53 minutes on a single 8 GB VRAM GPU, confirming the cost-effectiveness of lightweight LLMs for vulnerability detection.
6) **Instruction-tuning improves zero-shot performance:** Gemma-4-E2B-IT (instruction-tuned) achieves 3× higher accuracy than CodeGemma-2B (completion-only) in zero-shot mode, demonstrating that instruction alignment significantly benefits structured classification even without task-specific training.

C. Comparison with Existing Methods

To contextualise the results reported in Section IV-B, this subsection compares our multi-model SHARP-LLM framework against relevant baselines from the recent literature. All comparisons use the same task formulation — multi-class CWE classification on the NIST Juliet Test Suite for C/C++ v1.3 — and we adopt consistent metrics (macro-averaged precision, recall, F1, accuracy, and per-sample inference latency) wherever the original studies report them.

1) Baseline Methods: We compare against four categories of prior work:

1) **Traditional SAST tools** — rule-based static analysers such as Cppcheck, Flawfinder, and commercial SAST

scanners. Although widely deployed, these tools are known for high false-positive rates and limited generalisation to novel patterns [28].

2) **VulDeePecker [29]** — an early deep-learning approach that extracts *code gadgets* from program slices and trains a bidirectional LSTM for *binary* vulnerability detection (vulnerable vs. safe). It does not address multi-class CWE classification.
3) **Transformer baselines in the literature** — studies applying CodeBERT [23], GraphCodeBERT [24], and CodeT5 [21] to vulnerability detection. Most prior evaluations use *binary* classification on Devign or Big-Vul datasets with random splits; few attempt 118-class CWE classification with template-aware evaluation.

2) Discussion:

a) Advantage over binary detection methods.: Methods such as VulDeePecker [29] and the majority of Devign/Big-Vul evaluations perform *binary* classification (vulnerable vs. safe) — a fundamentally simpler task. Our framework tackles 118-way CWE classification, which is more actionable for developers because it identifies the *specific* weakness type, enabling targeted remediation. Despite the significantly harder task, our fine-tuned models achieve macro-F1 scores above 0.95, demonstrating that modern encoder architectures can handle fine-grained multi-class vulnerability categorisation effectively.

b) Advantage over rule-based SAST tools.: Traditional static analysers rely on hand-crafted rules and suffer from two well-documented limitations [28]: (1) high false-positive rates that erode developer trust, and (2) inability to generalise to novel coding patterns not covered by existing rules. Our learning-based approach achieves a macro false-positive rate below 3.4×10^{-5} per class (Table VI) while maintaining sub-30 ms inference latency, making it competitive with rule-based tools in speed but vastly superior in precision and adaptability.

c) Why our method performs better.: We attribute the strong performance of SHARP-LLM to three factors:

1) **Multi-model evaluation with controlled comparison.** By evaluating four encoder architectures under identical conditions, we identify CodeT5-Base as the optimal accuracy–efficiency trade-off, a finding obscured when only a single model is studied.
2) **Rigorous template-aware splitting.** Unlike random splits that inflate metrics through template leakage, our splitting strategy guarantees zero template overlap between train and test. The fact that all fine-tuned models still exceed 99.6% accuracy under this protocol demonstrates genuine generalisation rather than memorisation.
3) **Multi-paradigm coverage.** Including zero-shot generative models alongside fine-tuned encoders quantifies the value of task-specific training: even the best zero-shot model (Gemma-4-E2B-IT at 5.13 B parameters) underperforms the smallest fine-tuned model (CodeT5-Small at 35 M parameters) by **0.2545** in macro-F1, while requiring $23{,}000\times$ longer inference time. This result has practical implications for deployment decisions.

d) Conditions and limitations.: The comparison is subject to several caveats. First, all experiments use the Juliet Test Suite, which contains synthetic, template-generated code; performance on real-world vulnerability datasets (e.g., Devign, Big-Vul, ReVeal) may differ and remains a direction for future work. Second, the zero-shot models were evaluated on sampled subsets (50–500 samples) due to prohibitive inference costs; larger-scale zero-shot evaluation may yield different aggregate statistics. Finally, binary detection methods (VulDeePecker, Devign-based studies) solve a fundamentally different task, so cross-task metric comparisons should be interpreted with appropriate caution.

V. Conclusion and Future Work

This paper presented SHARP-LLM, a secure and lightweight framework for source code vulnerability detection and risk prioritization in healthcare software. Using fine-tuned large language models on the Juliet C/C++ Test Suite, the proposed framework demonstrated that compact, task-specific models can achieve strong multi-class CWE classification performance while maintaining practical computational requirements. In addition to vulnerability detection, SHARP-LLM introduced a healthcare-oriented post-detection pipeline that maps detected weaknesses to privacy and security risk categories, links them to policy-relevant control domains, and produces structured priority levels for remediation. The results indicate that lightweight fine-tuned models remain highly effective for source code vulnerability analysis and that the proposed risk prioritization layer improves the practical relevance of model outputs for healthcare software environments. Future work will focus on extending the framework to additional programming languages, validating the prioritization pipeline on real-world healthcare codebases, and exploring tighter integration with secure software development workflows.

A. Limitations

Despite the promising results, this work has several limitations. First, the experiments are based on the NIST Juliet Test Suite, which contains synthetic, template-generated code. Although the template-aware split reduces trivial pattern memorization, Juliet does not fully capture the complexity, diversity, multi-file dependencies, and coding practices of real-world software projects.

Second, the dataset is highly imbalanced, with a long-tailed CWE distribution. Several CWE categories contain very few samples, and some single-template categories are excluded from the test set, limiting the reliability of evaluation for rare classes. Low F1 scores for single-sample categories should therefore be interpreted as insufficient evaluation coverage rather than definitive model failure.

Third, all models use a fixed input length of 256 tokens. While this is suitable for the relatively short Juliet samples, it may truncate important contextual information in real-world source files where vulnerabilities span larger code regions or depend on broader program context.

B. Future Work

Future work will focus on improving generalizability and practical applicability. First, the framework can be extended with real-world vulnerability datasets such as Big-Vul, Devign, ReVeal, and D2A to evaluate performance beyond synthetic benchmarks and improve robustness in production-like settings.

Second, semantic-preserving code augmentation techniques, including variable renaming, dead code insertion, control-flow restructuring, and comment removal, can be applied to encourage models to learn vulnerability semantics rather than surface-level code patterns.

Third, cross-language and cross-project evaluation should be explored across languages such as Java, Python, and JavaScript. This would help assess whether the approach generalizes across different programming ecosystems and software projects.

Fourth, the CWE-to-privacy-threat mapping can be expanded to cover additional weakness categories, including cryptographic failures, authentication weaknesses, and supply-chain risks. This would improve the coverage of healthcare-oriented vulnerability prioritization.

Finally, integrating regulatory frameworks such as HIPAA, GDPR, and IEC 62443 can enable compliance-oriented reporting, strengthening the framework's value as a decision-support tool for healthcare and other safety-critical domains.

REFERENCES

[1] G. Dolcetti and E. Iotti, "A dual perspective review on large language models and code verification," *Frontiers in Computer Science*, vol. 7, Art. no. 1655469, 2025, doi: 10.3389/fcomp.2025.1655469.

[2] M. S. H. Shaon and M. S. Akter, "Modern approaches to software vulnerability detection: A survey of machine learning, deep learning, and large language models," *Electronics*, vol. 14, no. 22, Art. no. 4449, 2025, doi: 10.3390/electronics14224449.

[3] G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning," *Journal of Systems and Software*, vol. 212, Art. no. 112031, 2024, doi: 10.1016/j.jss.2024.112031.

[4] Y. Yang, X. Zhou, R. Mao, J. Xu, L. Yang, Y. Zhang, H. Shen, and H. Zhang, "DLAP: A deep learning augmented large language model prompting framework for software vulnerability detection," *Journal of Systems and Software*, vol. 219, Art. no. 112234, 2025, doi: 10.1016/j.jss.2024.112234.

[5] Z. Tian, M. Li, J. Sun, Y. Chen, and L. Chen, "Enhancing vulnerability detection by fusing code semantic features with LLM-generated explanations," *Information Fusion*, vol. 125, Art. no. 103450, 2026, doi: 10.1016/j.inffus.2025.103450.

[6] Q. Mao, Z. Li, X. Hu, K. Liu, X. Xia, and J. Sun, "Towards explainable vulnerability detection with large language models," *IEEE Transactions on Software Engineering*, vol. 51, no. 10, pp. 2957–2971, 2025, doi: 10.1109/TSE.2025.3605442.

[7] S. Chang, C. Geng, H. Huang, R. Wang, Q. Li, and Y. Zhang, "CodeSpeak: Improving smart contract vulnerability detection via LLM-assisted code analysis," *Journal of Systems and Software*, vol. 231, Art. no. 112635, 2026, doi: 10.1016/j.jss.2025.112635.

[8] J. Liu, G. Lin, H. Mei, F. Yang, and Y. Tai, "Enhancing vulnerability detection efficiency: An exploration of light-weight LLMs with hybrid code features," *Journal of Information Security and Applications*, vol. 88, Art. no. 103925, 2025, doi: 10.1016/j.jisa.2024.103925.

[9] A. Mechri, M. A. Ferrag, and M. Debbah, "SecureQwen: Leveraging LLMs for vulnerability detection in python codebases," *Computers & Security*, vol. 148, Art. no. 104151, 2025, doi: 10.1016/j.cose.2024.104151.

[10] F. Qiu, Z. Liu, B. Hu, Z. Cai, L. Bao, and X. Wang, "RLV: LLM-based vulnerability detection by retrieving and refining contextual information," *Journal of Systems and Software*, vol. 235, Art. no. 112756, 2026, doi: 10.1016/j.jss.2025.112756.

[11] T. Zheng, H. Liu, H. Xu, X. Chen, P. Yi, and Y. Wu, "FewVulD: A few-shot learning framework for software vulnerability detection," *Computers & Security*, vol. 144, Art. no. 103992, 2024, doi: 10.1016/j.cose.2024.103992.

[12] S. Zhang, Q. Wang, Q. Liu, E. Luo, and T. Peng, "VulTrLM: LLM-assisted vulnerability detection via AST decomposition and comment enhancement," *Empirical Software Engineering*, vol. 31, no. 1, 2025, doi: 10.1007/s10664-025-10738-7.

[13] M. A. Ferrag, A. Battah, N. Tihanyi, R. Jain, D. Maimut, F. Alwahedi, T. Lestable, N. S. Thandi, A. Mechri, M. Debbah, and L. C. Cordeiro, "SecureFalcon: Are we there yet in automated software vulnerability detection with LLMs?" *IEEE Transactions on Software Engineering*, vol. 51, no. 4, pp. 1248–1265, 2025, doi: 10.1109/TSE.2025.3530819.

[14] T. Hinrichs, E. Iannone, and R. Scandariato, "Beyond prompting: The role of phrasing tasks in vulnerability prediction for Java," *Cybersecurity*, vol. 8, Art. no. 111, 2025, doi: 10.1186/s42400-025-00476-0.

[15] C. D. Xuan, D. Q. Bui, and D. Q. Vu, "Large language models based vulnerability detection: How does it enhance performance?" *International Journal of Information Security*, vol. 24, Art. no. 69, 2025, doi: 10.1007/s10207-025-00983-8.

[16] S. M. Taghavi Far and F. Feyzi, "Large language models for software vulnerability detection: A guide for researchers on models, methods, techniques, datasets, and metrics," *International Journal of Information Security*, vol. 24, Art. no. 78, 2025, doi: 10.1007/s10207-025-00992-7.

[17] Y. Chen, Y. Huang, X. Chen, P. Shen, and L. Yun, "GPTVD: Vulnerability detection and analysis method based on LLM's chain of thoughts," *Automated Software Engineering*, vol. 33, no. 1, Art. no. 3, 2026, doi: 10.1007/s10515-025-00550-4.

[18] J. A. S. Ciavatta, J. R. B. Higuera, J. B. Higuera, J. A. S. Montalvo, T. S. Riera, and J. P. Melero, "Integration of large language models (LLMs) and static analysis for improving the efficacy of security vulnerability detection in source code," *Computers, Materials & Continua*, vol. 86, no. 3, pp. 1–40, 2026, doi: 10.32604/cmc.2025.074566.

[19] J. E. Ameh, A. Otebolaku, A. Shenfield, and A. Ikpehai, "C3-VULMAP: A Dataset for Privacy-Aware Vulnerability Detection in Healthcare Systems," *Electronics*, vol. 14, no. 13, p. 2703, 2025.

[20] NSA Center for Assured Software, "NIST SARD: Juliet Test Suite for C/C++," ver. 1.3, Zenodo, Oct. 1, 2017, doi: 10.5281/zenodo.4701387.

[21] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. 2021 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 8696–8708, doi: 10.18653/v1/2021.emnlp-main.685.

[22] Y. Wang, H. Le, A. Gotmare, N. Bui, J. Li, and S. C. H. Hoi, "CodeT5+: Open code large language models for code understanding and generation," in *Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 1069–1088.

[23] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[24] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, M. Duan, H. Svyatkovskiy, S. Fu, S. Tufano, M. Gong, N. Duan, D. Jiang, A. Cogo, C. de Wang, M. Zhou, and T. Liu, "GraphCodeBERT: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.

[25] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.

[26] CodeGemma Team, H. Zhao, J. Hui, J. Howland, N. Nguyen, S. Zuo, *et al.*, "CodeGemma: Open code models based on Gemma," *arXiv preprint arXiv:2406.11409*, 2024.

[27] Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, *et al.*, "Gemma: Open models based on Gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.

[28] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?," in *Proc. 2013 Int. Conf. Softw. Eng. (ICSE)*, 2013, pp. 672–681.

[29] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A deep learning-based system for vulnerability

detection," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2018, doi: 10.14722/ndss.2018.23165.
[30] R. Vinayakumar, K. P. Soman, P. Poornachandran, and V. K. Menon, "A deep-dive on machine learning for cyber security use cases: Principles, algorithms, and practices," in *Machine Learning for Computer and Cyber Security*, 2019.
[31] R. Vinayakumar *et al.*, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.
[32] M. H. Babu, R. Vinayakumar, and K. P. Soman, "A short review on applications of deep learning for cyber security," *arXiv preprint arXiv:1812.06247*, 2018.
[33] R. Vinayakumar *et al.*, "Robust intelligent malware detection using deep learning," *IEEE Access*, vol. 7, pp. 46717–46738, 2019.
[34] R. Vinayakumar *et al.*, "Application of deep learning architectures for cyber security," in *Machine Learning for Computer and Cyber Security*, 2019.
[35] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 508–512.
[36] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems*, 2019.
[37] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?," *IEEE Transactions on Software Engineering*, 2021.
[38] Y. Zheng *et al.*, "D2A: A dataset built for AI-based vulnerability detection methods using differential analysis," in *Proceedings of the IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice*, 2021.
[39] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.